



THE KNIFE

MANUALE TECNICO

Università degli Studi dell'Insubria – Laurea Triennale in Informatica

Progetto Laboratorio B

Creato da: Mattia Gerundino, Christian Pianarosa

SOMMARIO

INTRODUZIONE	2
OBIETTIVI DEL SISTEMA	2
LIBRERIE ESTERNE UTILIZZATE	2
USE CASE UML DIAGRAM	4
ARCHITETTURA DEL SISTEMA	5
Architettura generale	5
Pattern MVC utilizzato	6
Protocollo di comunicazione e stato della sessione	7
Modulo condiviso (DTO)	8
Modulo server	9
Modulo client.....	16
STRUTTURA DELLE CARTELLE DEL PROGETTO	24
Cartelle principali	24
Relazione tra risorse e codice	25
BASE DI DATI	25
Entità identificate	25
Relazioni e Cardinalità	25
Schema Concettuale ER (pre-ristrutturazione).....	0
Schema ER (post-ristrutturazione).....	0
Ristrutturazione dello Schema ER.....	1
Schema Relazionale	1
Vincoli e integrità dei dati.....	1
Utilizzo delle basi di dati nel sistema	1
STRUTTURE DATI.....	2
SCELTE ALGORITMICHE.....	2
TEST ESEGUITI	3
POSSIBILI SVILUPPI FUTURI	3
SITOGRAFIA/BIBLIOGRAFIA	4



INTRODUZIONE

THE KNIFE è un progetto realizzato per Laboratorio interdisciplinare A e B per il corso di laurea in informatica dell'Università degli Studi dell'Insubria anno 2025/2026.

È una piattaforma che consente di trovare ristoranti in tutto il mondo e selezionarli in base al luogo, alla tipologia del ristorante stesso, alla fascia di prezzo, alla possibilità o meno di prenotare un tavolo o di ordinare da asporto.

Questo documento descrive l'architettura tecnica dell'applicazione **TheKnife**, un sistema desktop sviluppato in **Java 21** con interfaccia grafica **JavaFX 21** e persistenza su database relazionale **PostgreSQL**. Il progetto è organizzato come un singolo modulo Maven (**project.demo**) che integra al proprio interno tre macro-componenti logiche ben distinte per responsabilità:

- un **modulo client**, costituito dalle viste FXML, dai relativi Controller JavaFX e dal componente di navigazione centralizzato (**Navigator**);
- un **modulo server** (nel senso di "livello di accesso ai dati"), costituito dalle classi del package db che incapsulano tutta la logica di connessione e interrogazione del database PostgreSQL tramite JDBC;
- un piccolo insieme di **classi condivise** (DTO/model), utilizzate sia dal client sia, indirettamente, dal livello di accesso ai dati per il trasporto delle informazioni tra i due strati.

Sebbene l'applicazione non sia un sistema client-server distribuito in senso stretto (client e "server" girano nello stesso processo Java, sulla stessa macchina), l'architettura del codice separa nettamente le responsabilità secondo un classico pattern a livelli (layered architecture): la UI non esegue mai query SQL direttamente, ma dialoga esclusivamente con un Data Access Object (DAO) che espone metodi ad alto livello (es. **cercaRistorante(...)**, **aggiungiPreferito(...)**, **rispostaRecensione(...)**).

OBIETTIVI DEL SISTEMA

Obiettivi generali: ricerca avanzata di ristoranti per città/cucina/prezzo/delivery/punteggio; registrazione e autenticazione di clienti e gestori; gestione preferiti; pubblicazione/modifica/cancellazione recensioni; CRUD ristoranti lato gestore; risposta alle recensioni; integrità dei dati tramite vincoli **SQL** e **PreparedStatement**.

Obiettivi qualitativi: separazione dei livelli di logica per manutenibilità ed estendibilità; modularità tramite DAO centralizzato; gestione robusta degli errori lato UI e backend; navigazione contestuale con caching dello stato di ricerca.

LIBRERIE ESTERNE UTILIZZATE

Tutte le dipendenze del progetto sono dichiarate nel file **pom.xml** e gestite tramite Apache Maven. La tabella seguente riassume le librerie esterne utilizzate, il loro ruolo nell'applicazione e la versione dichiarata.

Libreria	Versione	Ruolo nel progetto
org.postgresql:postgresql	42.7.3	Driver JDBC ufficiale per la connessione e l'interrogazione del database PostgreSQL da parte del livello DAO (package db).
org.openjfx:javafx-controls	21	Componenti grafici di base di JavaFX (Button, Label, TextField, ComboBox, TableView, ecc.) usati in tutte le viste dell'applicazione.



Libreria	Versione	Ruolo nel progetto
org.openjfx:javafx-fxml	21	Motore di caricamento e binding dei file FXML tramite <code>FXMLLoader</code> , con associazione automatica ai Controller annotati con <code>@FXML</code> .
org.openjfx:javafx-web	21	Motore WebView di JavaFX, incluso tra le dipendenze per eventuali visualizzazioni HTML/mappe embedded.
org.openjfx:javafx-swing	21	Livello di interoperabilità JavaFX/Swing, incluso come dipendenza di supporto.
org.openjfx:javafx-media	21	Supporto alla riproduzione di contenuti multimediali (audio/video) in JavaFX.
org.controlsfx:controlsfx	11.2.1	Libreria di componenti UI aggiuntivi per JavaFX (es. notifiche, controlli avanzati) non disponibili nel core.
com.dlsc.formsfx:formsfx-core	11.6.0	Framework per la costruzione dichiarativa di moduli/form complessi in JavaFX.
net.synedra:validatorfx	0.5.0	Libreria per la validazione dei campi di input nei form (es. controlli di formato su email, numeri, date).
org.kordamp.ikonli:ikonli-javafx	12.3.1	Set di icone vettoriali integrabili nei controlli JavaFX.
org.kordamp.bootstrapfx:bootstrapfx-core	0.4.0	Foglio di stile ispirato a Bootstrap per JavaFX, utilizzabile come base per lo stile dei controlli.
eu.hansolo:tilesfx	21.0.3	Libreria di componenti "tile" per dashboard e visualizzazioni sintetiche di dati (KPI, indicatori).
com.github.almasb:fxgl	17.3	Game engine/framework JavaFX incluso tra le dipendenze, potenzialmente utile per animazioni e interazioni avanzate dell'interfaccia.
org.junit.jupiter:junit-jupiter-api/-engine	5.10.2	Framework di unit testing (JUnit 5), dipendenza di scope <code>test</code> .



Le principali dipendenze di build (plugin Maven) sono invece:

Plugin	Versione	Ruolo
maven-compiler-plugin	3.13.0	Configura il compilatore Java per il livello di linguaggio 21 (source/target).
javafx-maven-plugin	0.0.8	Permette di avviare l'applicazione JavaFX tramite <code>mvn javafx:run</code> e di produrre immagini jlink personalizzate (launcher <code>app</code>).

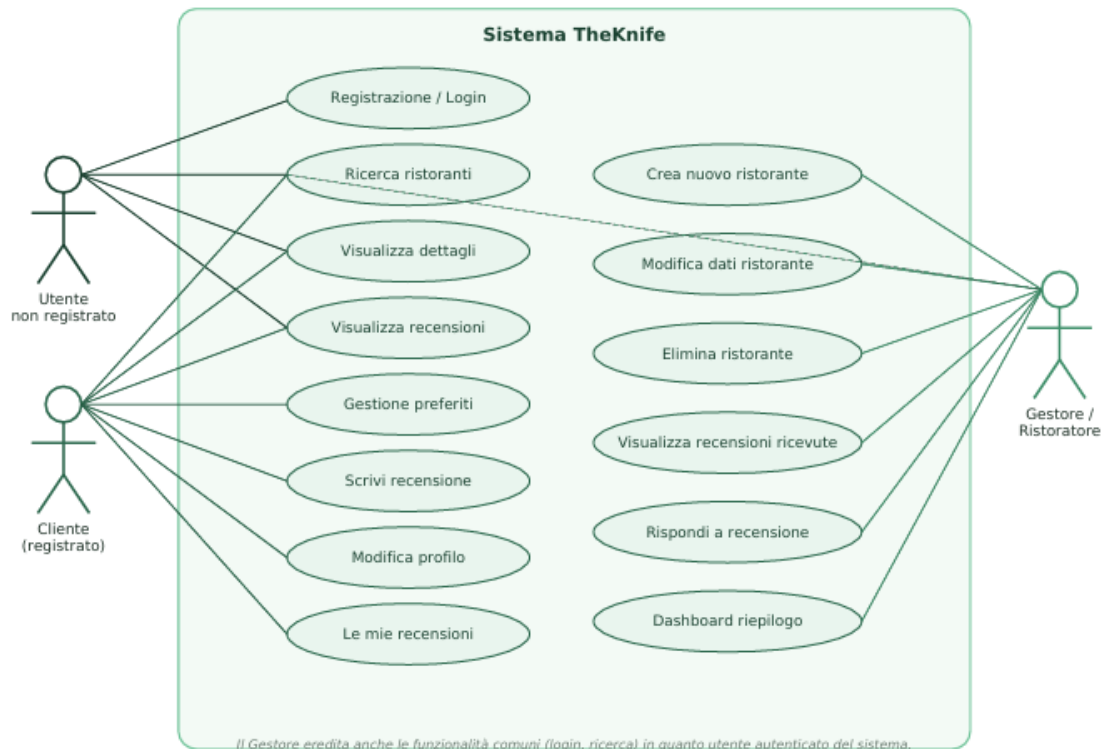
Nota architetturale: nonostante il ricco set di librerie UI disponibili (ControlsFX, FormsFX, ValidatorFX, Ikonli, BootstrapFX, TilesFX, FXGL), la maggior parte delle schermate attualmente implementate si basa sui controlli standard di `javafx.controls` e su un foglio di stile CSS personalizzato (`style.css`), che replica in JavaFX un linguaggio visivo simile a quello di un moderno sito web responsive (navbar, card, badge, chip, ecc.).

USE CASE UML DIAGRAM

Il diagramma seguente rappresenta i principali casi d'uso del sistema TheKnife e i tre attori che interagiscono con esso: l'**Utente non registrato** (visitatore anonimo), il **Cliente** (utente registrato con ruolo `cliente` nel database) e il **Gestore/Ristoratore** (utente registrato con ruolo `gestore`).

Attori e relativi casi d'uso principali:

- **Utente non registrato:** Registrazione/Login, Ricerca ristoranti, Visualizza dettagli, Visualizza recensioni
- **Cliente (registrato):** eredita i casi d'uso dell'utente non registrato e aggiunge Gestione preferiti, Scrivi recensione, Modifica profilo, Le mie recensioni
- **Gestore/Ristoratore:** eredita i casi d'uso comuni (login, ricerca) e aggiunge Crea nuovo ristorante, Modifica dati ristorante, Elimina ristorante, Visualizza recensioni ricevute, Rispondi a recensione, Dashboard riepilogo

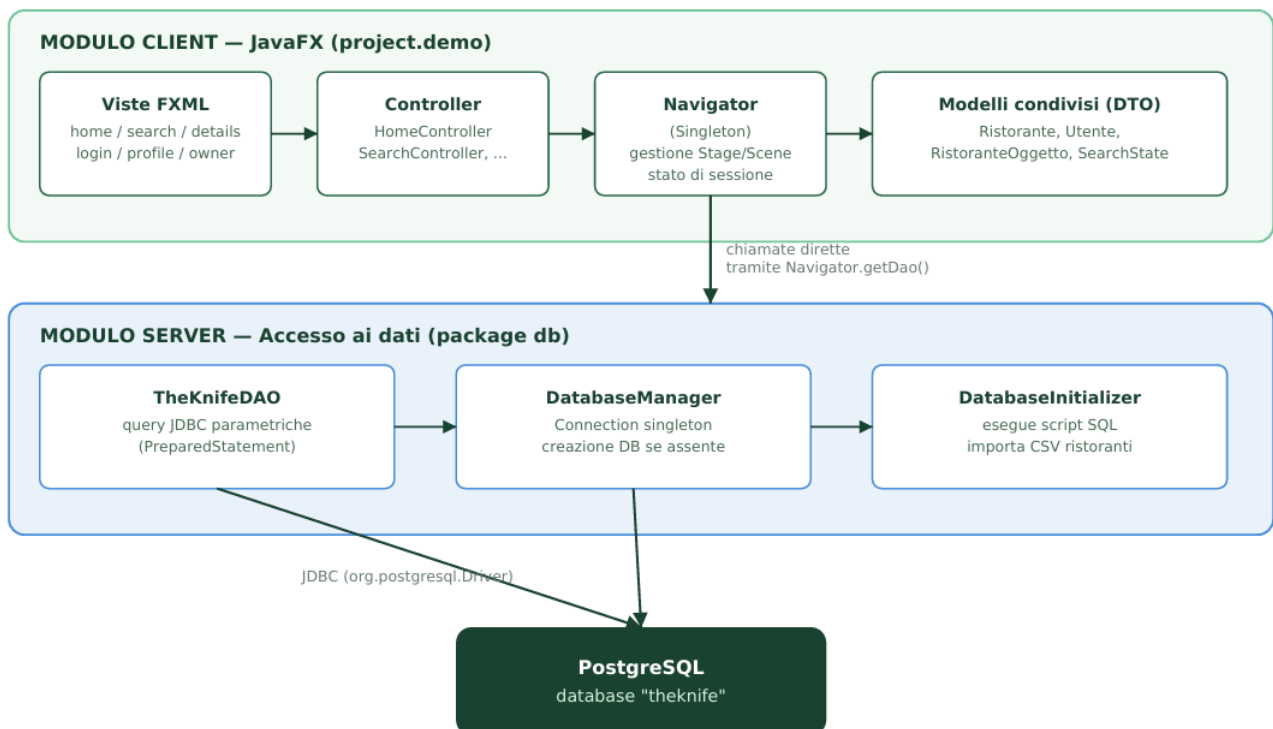


Le relazioni tra attori e casi d'uso rispecchiano fedelmente i vincoli implementati a livello di applicazione e di database:

- L'**Utente non registrato** può registrarsi/effettuare login, cercare ristoranti, visualizzare dettagli e recensioni già pubblicate, ma non può scrivere recensioni, salvare preferiti o gestire ristoranti.
- Il **Cliente** eredita tutte le funzionalità di consultazione e aggiunge: gestione dei preferiti, scrittura di recensioni, modifica del proprio profilo, consultazione delle proprie recensioni.
- Il **Gestore** dispone di casi d'uso esclusivi legati alla gestione dei ristoranti di sua proprietà: creazione, modifica ed eliminazione dei locali, consultazione delle recensioni ricevute e possibilità di rispondere pubblicamente a ciascuna di esse. Questi casi d'uso sono protetti sia a livello applicativo (i controller verificano il ruolo dell'utente loggato tramite `Navigator.getRuoloUtenteLoggato()`) sia a livello di database, tramite trigger dedicati.

ARCHITETTURA DEL SISTEMA

L'architettura di TheKnife segue un classico pattern a tre livelli (three-tier), pur essendo eseguita come singolo processo Java monolitico: modulo client (JavaFX), modulo server (accesso ai dati, package db) e database PostgreSQL, collegati tramite JDBC e raccordati dal Navigator, che espone alle viste sia lo Stage principale sia l'istanza condivisa del DAO.



Architettura generale

Descrizione della struttura a livelli

Il progetto adotta un'architettura a tre livelli:

1. presentazione: interfaccia JavaFX, controller e FXML;
2. logica applicativa: navigator, validazione, gestione di sessione;
3. persistenza: DAO, manager di connessione e script SQL.



Componenti principali

I principali componenti del sistema sono:

TheKnifeApp: entry point JavaFX e iniziatore dello stage;

Navigator: gestore singleton della navigazione e della sessione utente;

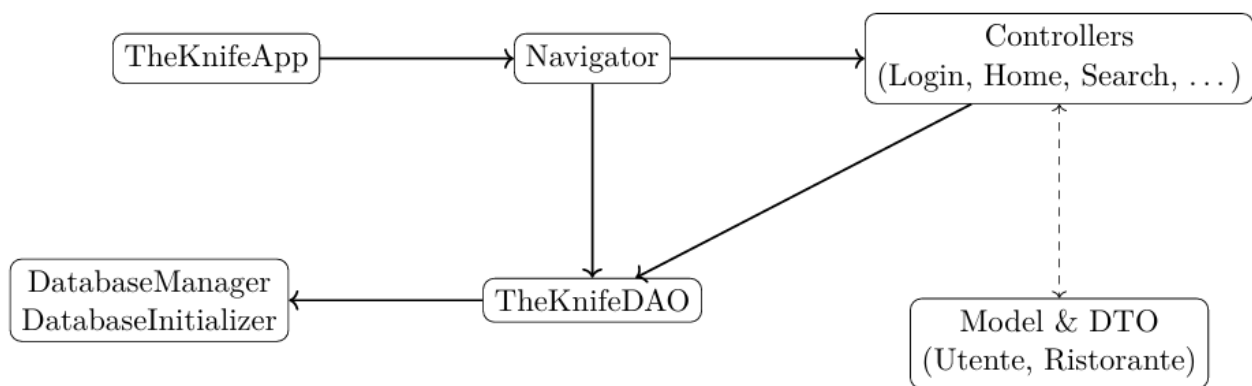
TheKnifeDAO: oggetto di accesso ai dati che incapsula query SQL e PreparedStatement;

DatabaseManager: manager di connessione JDBC e creazione del database;

DatabaseInitializer: inizializza schema e importa dati CSV;

Controller JavaFX: gestiscono l'interazione utente e invocano il DAO;

Model: classi come Utente e Ristorante che rappresentano i dati dell'applicazione.



Pattern MVC utilizzato

Motivazione dell'uso di MVC

Il pattern Model-View-Controller è appropriato per applicazioni JavaFX che separano chiaramente UI, rappresentazione dati e logica di controllo. In TheKnife, MVC facilita la gestione della complessità e riduce l'accoppiamento tra interfacce FXML e persistenza.

Mappatura delle responsabilità

- **Model:** classi Utente, Ristorante, e oggetti secondari come [SearchController](#). [RistoranteOggetto](#) rappresentano i dati.
- **View:** file FXML e CSS nella cartella [resources/project/demo](#) definiscono layout, stile e comportamenti statici.
- **Controller:** classi Java nella cartella [src/main/java/project/demo](#) gestiscono eventi, validazione e comunicazione con il DAO.



Esempio di separazione MVC

Nel controller `LoginController` la logica di validazione e le chiamate a `TheKnifeDAO` sono chiaramente separate dalla vista FXML.

```

1 @FXML
2 private void handleLogin() {
3     String email = loginUsername.getText().trim();
4     String passwordInserita = loginPassword.getText();
5     if (email.isEmpty() || passwordInserita.isEmpty()) {
6         showLoginError("Inserisci email e password!");
7         return;
8     }
9     if (!isValidEmail(email)) {
10        showLoginError("Inserisci un'email valida!");
11        return;
12    }
13    TheKnifeDAO dao = Navigator.getInstance().getDao();
14    try (ResultSet rs = dao.login(email)) {
15        if (rs.next()) {
16            String hashSalvato = rs.getString("password_hash");
17            int idUtente = rs.getInt("id_utente");
18            String ruolo = rs.getString("ruolo");
19            Navigator.getInstance().setIdUtenteLoggato(idUtente);
20            Navigator.getInstance().setRuoloUtenteLoggato(ruolo);
21            Navigator.getInstance().navigateTo("home-view-logged.
                fxml", "Home Cliente");
22        } else {
23            showLoginError("Email non trovata!");
24        }
25    } catch (SQLException e) {
26        showLoginError("Errore di connessione al database.");
27    }
28 }

```

Il **Navigator** (classe singleton) funge da elemento di raccordo tra i due macro-moduli: possiede un riferimento allo Stage principale di JavaFX (per il livello client) e un riferimento all'istanza di `TheKnifeDAO` collegata al database (per il livello server), esponendo entrambi ai vari Controller tramite i metodi `getInstance()`, `getDao()`.

Protocollo di comunicazione e stato della sessione

Non essendo un'applicazione distribuita, `TheKnife` non utilizza un vero protocollo di rete (HTTP, socket TCP personalizzati, ecc.) per la comunicazione tra client e server: il "protocollo di comunicazione" tra i due livelli logici è costituito da **chiamate a metodo dirette** (invocazioni Java sincrone) verso l'istanza condivisa di `TheKnifeDAO`, la quale internamente traduce ogni richiesta in una query SQL parametrica eseguita tramite **JDBC** sulla connessione a PostgreSQL gestita da `DatabaseManager`.

Ogni operazione segue lo stesso schema concettuale:

- Un Controller JavaFX intercetta un evento dell'interfaccia (click su un pulsante, submit di un form, ecc.) tramite un handler annotato `@FXML`;
- Il Controller richiama `Navigator.getInstance().getDao()` per ottenere il riferimento al DAO condiviso;
- Viene invocato il metodo del DAO corrispondente all'operazione desiderata (es. `cercaRistorante`, `aggiungiRecensione`, `rispostaRecensione`), passando i parametri come argomenti tipizzati Java (String, Integer, Double, Boolean...);
- Il DAO costruisce ed esegue una `PreparedStatement` JDBC, restituendo un `ResultSet` (per le query di lettura) oppure un intero che rappresenta il numero di righe modificate (per le operazioni di scrittura);
- Il Controller consuma il risultato e aggiorna la scena JavaFX corrente (es. ricostruendo dinamicamente le card dei risultati).



Gestione dello stato di sessione

Lo stato dell'utente autenticato è mantenuto interamente in memoria all'interno del `Navigator`, tramite due campi:

- `idUtenteLoggato`: identificativo numerico dell'utente autenticato (-1 se nessun utente ha effettuato l'accesso), impostato da `LoginController` dopo un login riuscito.
- `ruoloUtenteLoggato`: ruolo dell'utente autenticato (`cliente` o `gestore`), utilizzato dai vari controller per abilitare/disabilitare funzionalità riservate e per decidere quale schermata mostrare dopo il login.

Il metodo `Navigator.logout()` azzerava entrambi i campi, riportando l'applicazione allo stato "ospite". Non essendoci un vero canale di rete, non esiste un token di sessione: lo stato vive per l'intera durata del processo JavaFX e viene perso alla chiusura dell'applicazione, comportamento coerente con un'applicazione desktop single-user.

Un ulteriore stato applicativo condiviso è rappresentato dalla classe `SearchState`, che conserva i valori correnti dei filtri di ricerca (città, tipo cucina, range di prezzo, flag delivery/prenotazione, stelle minime) come campi statici, permettendo così a più controller di leggere/scrivere lo stato dei filtri senza doverlo esplicitamente propagare tra le schermate.

Modulo condiviso (DTO)

Le seguenti classi rappresentano i modelli/DTO (Data Transfer Object) condivisi tra i livelli client e server, utilizzati per trasportare i dati letti/scrritti sul database verso l'interfaccia utente e viceversa:

Classe	Descrizione
<code>Ristorante</code>	Modello "di dominio" immutabile di un ristorante (nome, città, nazione, indirizzo, coordinate, fascia di prezzo, tipo di cucina, delivery, prenotazione online), con getter pubblici e nessun setter.
<code>SearchController.RistoranteOggetto</code>	DTO "di presentazione" costruito direttamente a partire da un <code>ResultSet</code> JDBC (costruttore <code>RistoranteOggetto(ResultSet rs)</code>), usato per popolare le card dei risultati di ricerca; include anche i campi calcolati <code>media_stelle</code> (arrotondata a intero in <code>stelleIntere</code>) e le coordinate geografiche.
<code>Utente</code>	Modello di dominio di un utente registrato (nome, cognome, username, email, password, domicilio, data di nascita, tipo account), con getter e setter sui campi modificabili.
<code>SearchState</code>	Contenitore statico dello stato corrente dei filtri di ricerca, condiviso tra le schermate di ricerca (città, tipo cucina, range di prezzo, delivery, prenotazione online, stelle minime) con metodo <code>reset()</code> per riportarlo ai valori di default.

Nota di progetto: il DTO `RistoranteOggetto` è definito come classe interna statica di `SearchController` anziché come classe di primo livello: si tratta di una scelta progettuale che accoppia il modello di presentazione al controller che maggiormente lo utilizza, ma che viene comunque condiviso e riutilizzato da altri controller (ad es. `RestaurantDetailsController` e `Navigator`) tramite il suo nome completo `SearchController.RistoranteOggetto`.



Modulo server

Il "modulo server" comprende tutte le classi del package `db`, responsabili della connessione al database e dell'esecuzione delle query. Le classi principali sono:

Classe	Responsabilità
<code>DatabaseManager</code>	Gestisce la connessione JDBC a PostgreSQL (host, porta, nome database, credenziali) tramite una singola <code>Connection</code> condivisa (pattern Singleton semplificato); espone <code>initialize()</code> , che verifica/crea il database "theknife" se assente (<code>ensureDatabaseExists()</code>) e delega a <code>DatabaseInitializer</code> la creazione dello schema.
<code>DatabaseInitializer</code>	Verifica l'esistenza delle tabelle applicative; se assenti, esegue lo script <code>theknife_create_db.sql</code> (creazione di tabelle, indici e trigger) e successivamente importa il catalogo iniziale dei ristoranti da un file CSV incluso tra le risorse, con logica di sanificazione dei caratteri speciali (es. simbolo " " usato sia come separatore di campo sia, in alcuni casi, all'interno del nome del locale) e conversione tipizzata dei singoli campi (UUID, double, interi, booleani).
<code>TheKnifeDAO</code>	Responsabilità TheKnifeDAO è il Data Access Object centrale. Incapsula tutte le query SQL e fornisce metodi per autenticazione, ricerca, preferiti, recensioni e CRUD dei ristoranti. Attributi principali: • Connection conn - connessione al database; Metodi principali: TheKnifeDAO contiene oltre cinquanta metodi, tra cui: • <code>registrazione(...)</code>; • <code>login(String email)</code>; • <code>cercaRistorante(...)</code>; • <code>isPreferito(int idUtente, String idRistorante)</code>; • <code>aggiungiPreferito(...)</code>, <code>rimuoviPreferito(...)</code>; • <code>aggiungiRecensione(...)</code>, <code>modificaRecensione(...)</code>, <code>eliminaRecensione(...)</code>; • <code>aggiungiRistorante(...)</code>, <code>modificaRistorante(...)</code>, <code>eliminaRistorante(...)</code>; • <code>rispostaRecensione(...)</code>; • <code>updateProfiloUtente(...)</code>, <code>updateProfiloUtenteConPassword(...)</code>. Relazioni TheKnifeDAO è utilizzato da tutti i controller e da Navigator. Riceve la connessione da <code>DatabaseManager</code> e offre un'interfaccia uniforme per la persistenza.



Schema del database

Lo schema relazionale, definito in `theknife_create_db.sql`, è composto dalle seguenti tabelle principali:

Tabella	Descrizione sintetica
Utenti	Anagrafica di clienti e gestori, con vincolo CHECK sul campo ruolo ('cliente' o 'gestore') e vincoli geografici sulle coordinate di domicilio. Contiene attributi essenziali di autenticazione e anagrafica Attributi principali • nome, cognome, username, email, password; • domicilio, dataNascita, tipoAccount. Metodi principali Getter e setter standard per gestire i dati dell'utente. Autenticazione utenti Il flusso di registrazione passa attraverso <code>LoginController.handleRegister</code> e <code>TheKnifeDAO.registrazione</code>. Validazione Il controller esegue controlli su campi obbligatori, formato email e presenza della data di nascita. Le coordinate domicilio sono inizialmente mockate con valori di default. Persistenza La query di registrazione utilizza <code>PreparedStatement</code> per prevenire SQL injection. <pre> 1 String sql = "" 2 INSERT INTO Utenti 3 (nome, cognome, email, password_hash, data_nascita, 4 luogo_domicilio, lat_domicilio, lon_domicilio, ruolo 5) 6 VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?) 7 "" 8 try (PreparedStatement ps = conn.prepareStatement(sql)) { 9 ps.setString(1, nome); 10 ps.setString(2, cognome); 11 ps.setString(3, email); 12 ps.setString(4, passwordHash); 13 ps.setDate(5, dataNascita); 14 ps.setString(6, luogoDomicilio); 15 ps.setDouble(7, latDomicilio); 16 ps.setDouble(8, lonDomicilio); 17 ps.setString(9, ruolo); 18 return ps.executeUpdate(); 19 }</pre>



Tabella	Descrizione sintetica
	Flusso di login Il login cerca l'utente per email e recupera <code>password_hash</code> e ruolo. Il controllo dell'hash non è ancora pienamente implementato; viene utilizzato un confronto testuale nel codice attuale. Gestione dei ruoli Il ruolo <code>CLIENTE</code> e <code>GESTORE</code> determina l'interfaccia mostrata all'utente. <code>Navigator</code> e i controller rimandano alle viste appropriate.
<code>RistorantiTheKnife</code>	Catalogo dei ristoranti; la chiave primaria <code>id_ristorante</code> è di tipo <code>VARCHAR(50)</code> per ospitare gli UUID provenienti dal dataset CSV di importazione. Include un riferimento opzionale (<code>id_gestore</code>) al proprietario del locale.
<code>Recensioni</code>	Recensioni dei clienti sui ristoranti, con vincolo <code>UNIQUE(id_ristorante, id_utente)</code> che garantisce al massimo una recensione per cliente per locale, e vincolo <code>CHECK</code> sul range delle stelle (1-5).
<code>RisposteRecensioni</code>	Risposte dei gestori alle recensioni, con vincolo <code>UNIQUE(id_recensione)</code> che impedisce più di una risposta per recensione.
<code>Preferiti</code>	Tabella associativa molti-a-molti tra <code>Utenti</code> e <code>RistorantiTheKnife</code> , con chiave primaria composta.

L'integrità referenziale e i vincoli di dominio sono rinforzati da tre **trigger** PL/pgSQL:

- `tr_check_ruolo_gestore`: impedisce che un ristorante venga associato, come `id_gestore`, a un utente il cui ruolo non sia 'gestore';
- `tr_check_ruolo_cliente`: impedisce che una recensione venga associata a un utente il cui ruolo non sia 'cliente';
- `tr_check_gestore_risposta`: impedisce che un gestore risponda a una recensione relativa a un ristorante che non gli appartiene, confrontando l'`id_gestore` proposto con quello effettivo del ristorante recensito.

Metodi principali esposti da TheKnifeDAO

Metodo	Descrizione
<code>registrazione(...)</code>	Inserisce un nuovo utente (cliente o gestore).
<code>login(email)</code>	Recupera id, hash password e ruolo dato l'indirizzo email.
<code>cercaRistorante(...)</code>	Logica di ricerca La ricerca è implementata in <code>TheKnifeDAO.cercaRistorante</code>. Gestisce filtri combinati: città, tipo di cucina, prezzo minimo/max, delivery, prenotazione online e rating minimo.



Metodo	Descrizione
	Querydinamica Il metodo costruisce un <code>StringBuilder</code> e una lista di parametri per generare una query SQL flessibile. <pre> 1 StringBuilder sql = new StringBuilder("") 2 SELECT r.*, COALESCE(AVG(rec.stelle), 0) AS media_stelle, 3 COUNT(rec.id_recensione) AS num_recensioni 4 FROM RistorantiTheKnife r 5 LEFT JOIN Recensioni rec ON rec.id_ristorante = r. 6 id_ristorante 7 WHERE (?:text IS NULL OR ?:text = '' OR LOWER(r.citta) 8 LIKE LOWER(?)) 9 """); 10 11 if (tipoCucina != null && !tipoCucina.isEmpty()) { 12 sql.append(" AND (LOWER(r.tipologia_cucina) LIKE ? OR LOWER(r. 13 .nome) LIKE ?)"); 14 params.add("%" + tipoCucina.toLowerCase() + "%"); 15 params.add("%" + tipoCucina.toLowerCase() + "%"); 16 }</pre> Ordine e paginazione La vista riceve risultati ordinati per rating decrescente. <code>SearchController</code> applica inoltre ordinamenti lato client per prezzo e alfabetico.
<code>visualizzaRecensioni(idRistorante)</code>	Recupera le recensioni pubbliche di un ristorante con eventuale risposta del gestore.
<code>ristorantiVicini(lat, lon)</code>	Restituisce i 20 ristoranti più vicini a una coppia di coordinate, ordinati per distanza euclidea approssimata.
<code>aggiungiPreferito / rimuoviPreferito /visualizzaPreferiti(idUtente)</code>	Gestiscono l'aggiunta/rimozione di un ristorante dai preferiti di un cliente. Modello dei preferiti La tabella Preferiti usa una chiave composta (<code>id_utente</code>, <code>id_ristorante</code>) e assicura che un ristorante non sia duplicato due volte per lo stesso utente. Operazioni • <code>isPreferito</code> verifica l'esistenza; • <code>aggiungiPreferito</code> inserisce la relazione con ON CONFLICT DO NOTHING; • <code>rimuoviPreferito</code> elimina il preferito; • <code>visualizzaPreferiti</code> recupera i ristoranti preferiti con statistiche aggregate.



Metodo	Descrizione
	Elenco dei ristoranti preferiti di un cliente con media stelle e conteggio recensioni. <pre> 1 SELECT r.*, COALESCE(AVG(rec.stelle), 0) AS media_stelle, 2 COUNT(rec.id_recensione) AS num_recensioni 3 FROM Preferiti p 4 JOIN RistorantiTheKnife r ON r.id_ristorante = p.id_ristorante 5 LEFT JOIN Recensioni rec ON rec.id_ristorante = r.id_ristorante 6 WHERE p.id_utente = ? 7 GROUP BY r.id_ristorante 8 ORDER BY r.nome; </pre>
aggiungiRecensione / modificaRecensione / eliminaRecensione	La tabella Recensioni contiene il punteggio stelle, il testo e il timestamp <code>data_recensione</code>. Prevede un vincolo unico per evitare doppie recensioni dello stesso utente su uno stesso locale. Inserimento recensione <code>aggiungiRecensione</code> esegue l'inserimento con <code>PreparedStatement</code>. <pre> 1 String sql = "" 2 INSERT INTO Recensioni (id_ristorante, id_utente, stelle, 3 testo) 4 VALUES (?, ?, ?, ?) 5 ""; 6 try (PreparedStatement ps = conn.prepareStatement(sql)) { 7 ps.setString(1, idRistorante); 8 ps.setInt(2, idUtente); 9 ps.setInt(3, stelle); 10 ps.setString(4, testo); 11 return ps.executeUpdate(); 12 } </pre> Modifica e cancellazione Le operazioni <code>modificaRecensione</code> ed <code>eliminaRecensione</code> includono condizioni su <code>id_utente</code> in modo da autorizzare solo l'autore.
mieRecensioni(idUtente)	Elenco delle recensioni scritte da un cliente, con ristorante e risposta del gestore associati. <pre> 1 SELECT u.nome, u.cognome, rec.stelle, rec.testo, rec. 2 data_recensione, 3 risp.testo AS risposta_gestore, risp.data_risposta 4 FROM Recensioni rec 5 JOIN Utenti u ON u.id_utente = rec.id_utente 6 LEFT JOIN RisposteRecensioni risp ON risp.id_recensione = rec. 7 id_recensione 8 WHERE rec.id_ristorante = ? 9 ORDER BY rec.data_recensione DESC; </pre>
aggiungiRistorante(...)	Inserisce un nuovo ristorante associandolo al gestore che lo crea. I gestori possono aggiungere ristoranti tramite <code>NewRestaurantController</code> e <code>TheKnifeDAO.aggiungiRistorante</code>.



Metodo	Descrizione
	<pre> 1 String sql = "" 2 INSERT INTO RistorantiTheKnife 3 (id_ristorante, nome, nazione, citta, indirizzo, 4 latitudine, longitudine, 5 prezzo_medio, delivery, prenotazione_online, 6 tipologia_cucina, id_gestore) 7 VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?) 8 "" 9 10 try (PreparedStatement ps = conn.prepareStatement(sql)) { 11 ps.setString(1, idRistorante); 12 ps.setString(2, nome); 13 ... 14 return ps.executeUpdate(); 15 }</pre>
visualizzaRiepilogo(idGestore)	Riepilogo dei ristoranti di un gestore con media stelle e numero di recensioni.
visualizzaRecensioniGestore(idGestore)	Tutte le recensioni ricevute dai ristoranti di un gestore, con indicazione se già risposte.
rispostaRecensione(...)	Inserisce la risposta di un gestore a una recensione, protetta dal trigger tr_check_gestore_risposta per assicurare che solo il gestore proprietario del ristorante possa rispondere. I gestori rispondono alle recensioni dall'interfaccia OwnerDashboardController. Il metodo rispostaRecensione registra la risposta nella tabella RisposteRecensioni. Esempio SQL risposta <pre> 1 INSERT INTO RisposteRecensioni (id_recensione, id_gestore, testo) 2 VALUES (?, ?, ?);</pre>

Dettaglio delle classi server

[DatabaseManager](#) e [DatabaseInitializer](#) queste due classi meritano un approfondimento specifico poiché incorporano scelte implementative e limiti tecnici rilevanti ai fini della manutenibilità del sistema.

DatabaseManager *package db — gestione connessione JDBC*

Responsabilità

DatabaseManager gestisce la connessione JDBC verso il database PostgreSQL. Si occupa anche di verificare l'esistenza del database e di creare il database iniziale se manca.

Metodi principali

- **getConnection()** - restituisce una connessione Connection singleton;
- **initialize()** - avvia la creazione del database e lo schema se necessario;
- **ensureDatabaseExists()** - controlla l'esistenza del database e lo crea.



DatabaseInitializer package db — creazione schema e import dataset

Responsabilità

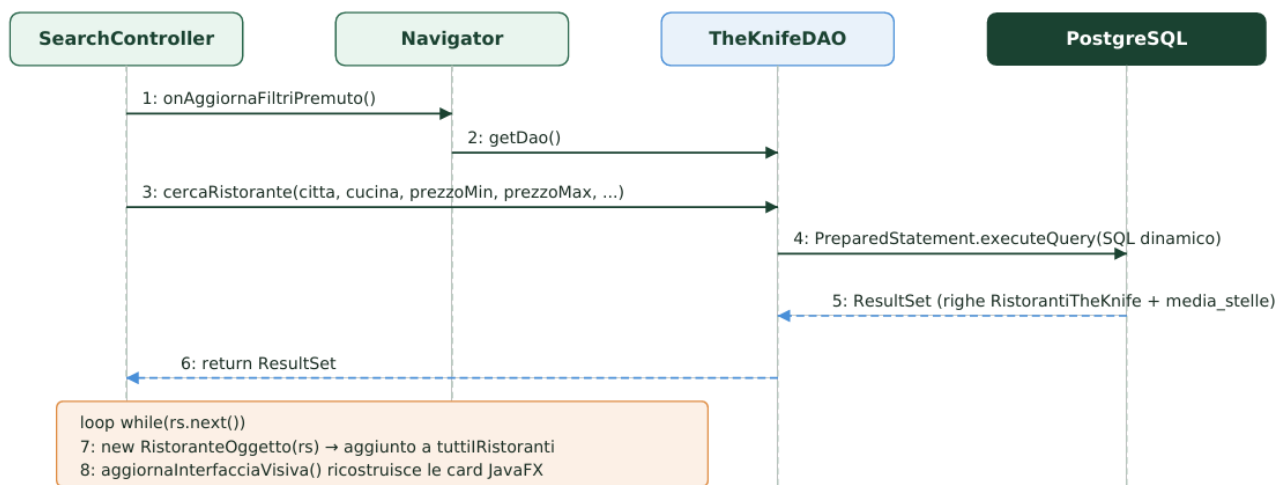
DatabaseInitializer verifica se lo schema e le tabelle sono presenti. Se non lo sono, esegue lo script SQL di creazione e importa i dati CSV iniziali.

Metodi principali

- **initializeIfNeeded()** - inializza il database se le tabelle mancano;
- **executeSqlScript(Connection conn, String resourcePath)** - esegue lo script SQL;
- **importaRistoranti(Connection conn, String resourcePath)** - importa dati da CSV con sanitizzazione.

Diagramma di sequenza Server

Il diagramma seguente illustra il flusso di una tipica operazione di ricerca ristoranti, dal click dell'utente sul pulsante "Aggiorna" fino al rendering dei risultati, evidenziando il ruolo del "modulo server" (**TheKnifeDAO** e PostgreSQL).



- 1. SearchController: onAggiornaFiltriPremuto()
- 2. SearchController → Navigator: getDao()
- 3. SearchController → TheKnifeDAO: cercaRistorante(citta, cucina, prezzoMin, prezzoMax, ...)
- 4. TheKnifeDAO → PostgreSQL: PreparedStatement.executeQuery(SQL dinamico)
- 5. PostgreSQL → TheKnifeDAO: ResultSet (righe RistorantiTheKnife + media_stelle)
- 6. TheKnifeDAO → SearchController: return ResultSet
- 7. loop while(rs.next()): new RistoranteOggetto(rs) → aggiunto a tuttilRistoranti
- 8. aggiornaInterfacciaVisiva() ricostruisce le card JavaFX
- 9. rendering dei risultati e della paginazione nella VBox containerRisultati



Modulo client

Il "modulo client" comprende le viste FXML, i rispettivi Controller JavaFX e il componente **Navigator**, responsabile della navigazione tra le schermate. Le classi principali sono:

Classe	Responsabilità
TheKnifeApp	Entry point dell'applicazione (estende <code>javafx.application.Application</code>); inizializza il Navigator con lo Stage principale e carica la prima schermata pubblica.
Navigator	Singleton che centralizza la navigazione tra scene FXML (<code>navigateTo</code>, <code>navigateToSearchWithQuery</code>, <code>navigateToSearchWithAdvancedFilters</code>, <code>navigateToRestaurantDetails</code>, <code>backToSearchResults</code>), lo stato di sessione dell'utente autenticato, risolve i percorsi FXML e il riferimento al DAO condiviso. Implementa inoltre un meccanismo di caching della schermata di ricerca (<code>cachedSearchView</code>), che permette di tornare istantaneamente ai risultati precedenti senza dover ripetere la query. Navigator dipende da <code>DatabaseManager</code>, <code>TheKnifeDAO</code> e <code>controller FXML</code>. Fornisce i dati e il contesto di sessione ai controller e funge da orchestratore tra UI e modello. Attributi principali: • Stage stage - riferimento allo stage primario; • TheKnifeDAO dao - istanza DAO condivisa; • int idUtenteLoggato - id dell'utente autenticato; • String ruoloUtenteLoggato - ruolo CLIENTE o GESTORE; • Parent cachedSearchView e String cachedSearchTitle - cache della vista di ricerca. Metodi principali: • getInstance() - ritorna il singleton; • navigateTo(String fxmlFile, String title) - carica e mostra una vista; • navigateToHome() e navigateToHomeIntelligent() - scegli la home corretta in base al ruolo; • navigateToProfile() - gestisce la navigazione al profilo utente; • navigateToSearchWithQuery(String queryTesto) - passa i parametri di ricerca a <code>SearchController</code>; • navigateToRestaurantDetails(SearchController.RistoranteOggetto ristorante) - passa dati alla vista dei dettagli; • logout() - resetta lo stato di sessione.



Classe	Responsabilità
HomeController / HomeLoggedInController	Gestiscono rispettivamente la home page pubblica e quella per utenti autenticati: ricerca globale, scelta rapida per categoria di cucina, pannello filtri avanzati.
LoginController	Gestisce sia il login sia la registrazione di nuovi utenti, smistando l'utente autenticato verso la home cliente o la dashboard gestore in base al ruolo letto dal database.
SearchController	Controller più complesso dell'applicazione: gestisce i filtri di ricerca, l'esecuzione dinamica delle query tramite il DAO, la costruzione manuale (via codice Java) delle card dei risultati e la paginazione lato client dei risultati già recuperati. Responsabilità • inizializzazione delle liste di filtri e dei menu; • invocazione del metodo <code>cercaRistorante</code> del DAO; • gestione di paginazione e risultati dinamici; • gestione preferiti lato interfaccia e toggle preferito. Classe interna di supporto <code>SearchController.RistoranteOggetto</code> è una classe interna che incapsula i risultati della query e semplifica il rendering delle card.
RestaurantDetailsController	gestisce la scheda dettagliata di un ristorante, comprese le recensioni e la pubblicazione di nuove recensioni. Responsabilità • visualizzazione dettagli ristorante e rating; • selezione votazione a stelle; • inserimento recensioni lato cliente con controllo di sessione; • gestione di accesso a preferiti e ricerca.
CustomerProfileController	Gestisce la visualizzazione e la modifica del profilo del cliente autenticato. Responsabilità • caricamento dati utente da <code>TheKnifeDAO.getDatUtente</code>; • aggiornamento profilo con o senza password; • gestione del logout e della navigazione verso ricerca e preferiti Esempio di update profilo



Classe	Responsabilità
	<pre> 1 boolean successo; 2 if (!nuovaPassword.isEmpty()) { 3 successo = dao.updateProfiloUtenteConPassword(4 idUtenteLoggato, 5 nome, 6 cognome, 7 dataSql, 8 domicilio, 9 0.0, 10 0.0, 11 nuovaPassword 12); 13 } else { 14 successo = dao.updateProfiloUtente(15 idUtenteLoggato, 16 nome, 17 cognome, 18 dataSql, 19 domicilio, 20 0.0, 21 0.0 22); 23 }</pre>
OwnerDashboardController	fornisce la dashboard per il gestore. Visualizza i ristoranti proprietà, le recensioni del proprio locale e permette l'aggiornamento dei dati. Responsabilità • caricamento riepilogo ristoranti e recensioni da visualizzaRiepilogo e visualizzaRecensioniGestore; • apertura e modifica dei ristoranti gestiti; • risposta alle recensioni e visualizzazione KPI; • eliminazione dei ristoranti con conferma. Esempio di risposta alle recensioni <pre> 1 int updated = dao.rispostaRecensione(r.idRecensione, idGestore, 2 testoRisposta); 3 if (updated > 0) { 4 caricaRistorantiERecensioni(); 5 Optional<RistoranteSummary> opt = ristoranti.stream() 6 .filter(x -> x.id.equals(selectedRestaurantId)).findFirst(); 7 opt.ifPresent(this::openRestaurant); 8 }</pre>
ReviewsController:	mostra le recensioni lasciate dall'utente. Il controller visualizza ogni recensione in schede dinamiche e supporta la navigazione verso altre aree.



Classe	Responsabilità
	Responsabilità • caricamento recensioni personali con <code>getRecensioniUtente</code>; • rendering grafico delle stelle e delle card; • visualizzazione di una pagina alternativa se non ci sono recensioni.
<code>FavoritesController:</code>	gestisce la vista dei preferiti. Recupera i ristoranti preferiti dell'utente e costruisce una galleria dinamica. Responsabilità • caricamento preferiti con <code>getPreferitiUtente</code>; • rendering di card con nome, città e tipo di cucina; • fallback su vista vuota quando la lista è priva di elementi
<code>NewRestaurantController:</code>	gestisce il form di creazione di un nuovo ristorante da parte del gestore, invocando <code>aggiungiRistorante(...)</code> del DAO.
<code>MyRestaurantController:</code>	gestisce la modifica dei dati di un ristorante esistente da parte del gestore, invocando <code>modificaRistorante(...)</code> del DAO.

Meccanismo di navigazione

Singleton Navigator: L'applicazione adotta Navigator come singleton per centralizzare la gestione dello stage e dei percorsi delle viste. Questo approccio evita duplicazioni e mantiene coerente la logica di navigazione.

Risoluzione dinamica FXML

Il metodo `risolviPercorsoFXML` ricerca il file FXML nelle cartelle `NotLoggedInUser`, `LoggedInUser` e `OwnerUser`.

```

1 private URL risolviPercorsoFXML(String fxmlFile) {
2     String nomeFilePuro = fxmlFile.substring(fxmlFile.lastIndexOf
        ("/") + 1);

3     String[] tentativiPercorso = {
4         "/project/demo/NotLoggedInUser/" + nomeFilePuro,
5         "/project/demo/LoggedInUser/" + nomeFilePuro,
6         "/project/demo/OwnerUser/" + nomeFilePuro
7     };
8     for (String percorso : tentativiPercorso) {
9         URL urlTrovato = getClass().getResource(percorso);
10        if (urlTrovato != null) {
11            return urlTrovato;
12        }
13    }
14    return null;
15 }

```



Navigazione intelligente

Navigator determina la vista di destinazione in base allo stato di autenticazione e al ruolo. Questo supporta sia il flusso pubblico che quello autenticato.

Caching dello stato di ricerca

Per migliorare l'esperienza utente, Notifier salva la vista di ricerca e il titolo corrente nel caso in cui l'utente navighi ai dettagli del ristorante e poi ritorni indietro.

```

1 public void backToSearchResults() {
2     if (cachedSearchView != null) {
3         updateSceneRoot(cachedSearchView, cachedSearchTitle);
4     } else {
5         navigateTo("search-view.fxml", "Cerca");
6     }
7 }

```

Tutte le transizioni di schermata passano attraverso il **Navigator**, che utilizza un **FXMLLoader** per caricare il file FXML richiesto e, quando necessario, recupera il relativo Controller tramite `loader.getController()` per invocare metodi di inizializzazione personalizzati (ad esempio `inizializzaRicercaGlobale(...)` o `caricaDatiRistorante(...)`) prima di mostrare effettivamente la nuova schermata. La sostituzione della schermata avviene tramite `scene.setRoot(root)` quando una **Scene** è già stata creata, evitando così di distruggere e ricreare la finestra a ogni navigazione.

Dettaglio dei Controller

TheKnifeApp *entry point dell'applicazione*

Il metodo `start(Stage)` richiama `Navigator.getInstance()`, che innesca "a cascata" l'intero processo di bootstrap: costruttore privato → `DatabaseManager.initialize()` → eventuale creazione dello schema e import del CSV → creazione dell'istanza di `TheKnifeDAO`. Tutta l'inizializzazione del database avviene quindi in modo lazy al primo utilizzo del **Navigator**, e non in un metodo di bootstrap esplicito e separato.

Incoerenza nei path delle risorse. La chiamata `navigateTo("NotLoggedInUser/home-view.fxml", ...)` include il prefisso di cartella `NotLoggedInUser/`, mentre altre chiamate a `navigateTo` nel resto del codice (ad esempio in `SearchController.goToHome`) usano `"home-view.fxml"` senza alcun prefisso di cartella. Questa discrepanza nei percorsi delle risorse andrebbe verificata in fase di build/packaging, per assicurarsi che entrambe le forme risolvano correttamente il file FXML a runtime.

- **start(Stage stage):** setta lo stage in **Navigator** e carica `home-view.fxml`;
- **main(String[] args):** Lancia l'applicazione JavaFX con `launch()`.

```

1 public class TheKnifeApp extends Application {
2     @Override

```



```

3     public void start(Stage stage) {
4         Navigator.getInstance().setStage(stage);
5         Navigator.getInstance().navigateTo("NotLoggedInUser/home-
           view.fxml", "Trova il tuo ristorante");
6     }
7     public static void main(String[] args) {
8         launch();
9     }
10 }

```

HomeController *home page pubblica*

HomeController gestisce la home page pubblica, le ricerche globali e i filtri avanzati. Invia richieste a Navigator per caricare la **search-view** con i parametri corretti.

Responsabilità

- raccolta input utente per query testuale e filtri;
- navigazione verso la pagina dei risultati;
- supporto dei segmenti di ricerca rapida per categorie di cucina.

LoginController *autenticazione e registrazione*

Responsabilità

- validazione email e campi obbligatori;
- login e registrazione con gestione degli errori SQL;
- navigazione verso home e profilo in base al ruolo.

Esempio di gestione login

```

1 try (ResultSet rs = dao.login(email)) {
2     if (rs.next()) {
3         String hashSalvato = rs.getString("password_hash");
4         int idUtente = rs.getInt("id_utente");
5         String ruolo = rs.getString("ruolo");
6         Navigator.getInstance().setIdUtenteLoggato(idUtente);
7         Navigator.getInstance().setRuoloUtenteLoggato(ruolo);
8         if ("CLIENTE".equalsIgnoreCase(ruolo)) {
9             Navigator.getInstance().navigateTo("home-view-logged-
               fxml", "Home Cliente");
10        } else if ("GESTORE".equalsIgnoreCase(ruolo)) {
11            Navigator.getInstance().navigateTo("home-view-owner-
               fxml", "Home Ristoratore");
12        }
13    } else {
14        showLoginError("Email non trovata!");
15    }
16 }

```



Relazioni

`LoginController` collabora con `Navigator` e sfrutta il DAO per l'accesso ai dati. Non accede direttamente al database, rispettando il principio di separazione delle responsabilità.

`RestaurantDetailsController` *scheda di dettaglio ristorante (ospite e loggato)*

Una singola classe serve **due viste FXML diverse** (`restaurant-details-view.fxml` per gli ospiti e `restaurant-details-logged-view.fxml` per gli utenti autenticati): molti campi `@FXML` risultano quindi `null` a seconda della vista effettivamente caricata, e ogni utilizzo è infatti protetto difensivamente da un controllo `if (campo != null)`.

Funzionalità non ancora persistenti. I metodi `handlePubblicaRecensione()` e `handlePrenotaOra()` sono, allo stato attuale, degli stub: il primo registra un messaggio di log e svuota il campo di testo.

`OwnerDashboardController` *dashboard del gestore/ristoratore*

Utilizza una classe interna privata `Recensione`, un DTO di presentazione locale distinto dalle righe restituite da `TheKnifeDAO.visualizzaRecensioniGestore(...)`.

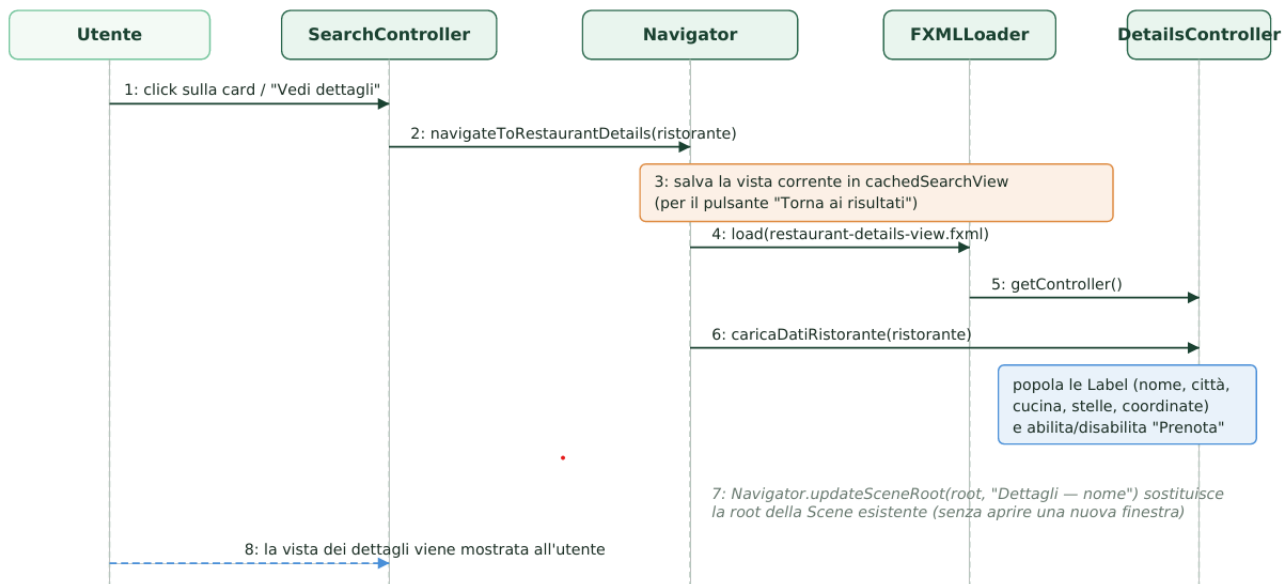
Dati di esempio hardcoded. Il metodo `initialize()` richiama `caricaDatiEsempio()`, che popola la lista con **due recensioni di esempio scritte staticamente nel codice**, anziché interrogare il DAO per il gestore realmente autenticato. Il rendering avviene poi correttamente all'interno di `Platform.runLater(...)`, pattern necessario per evitare che il container FXML risulti ancora `null` al momento della costruzione delle card.

Nota di sintesi: Il quadro che emerge da questi Controller è quello di un front-end JavaFX già completo dal punto di vista **visivo e di navigazione** con diverse funzionalità di scrittura (salvataggio profilo, pubblicazione recensioni, risposta alle recensioni, home autenticata)



Diagramma di sequenza Client

Il diagramma seguente mostra il flusso di navigazione client-side quando un utente, dalla pagina dei risultati di ricerca, clicca su un ristorante per visualizzarne la scheda di dettaglio, includendo il meccanismo di caching della vista di ricerca che consente il successivo ritorno immediato ai risultati.



- 1. Utente → SearchController: click sulla card / "Vedi dettagli"
- 2. SearchController → Navigator: navigateToRestaurantDetails(ristorante)
- 3. Navigator: salva la vista corrente in cachedSearchView (per il pulsante "Torna ai risultati")
- 4. Navigator → FXMLLoader: load(restaurant-details-view.fxml)
- 5. Navigator → DetailsController: getController()
- 6. Navigator → DetailsController: caricaDatiRistorante(ristorante) — popola le Label (nome, città, cucina, stelle, coordinate) e abilita/disabilita "Prenota"
- 7. Navigator.updateSceneRoot(root, "Dettagli — nome") sostituisce la root della Scene esistente (senza aprire una nuova finestra)
- 8. la vista dei dettagli viene mostrata all'utente

La stessa logica di caching viene sfruttata dal pulsante "← Torna ai risultati" presente nella schermata di dettaglio: invocando `Navigator.backToSearchResults()`, la root precedentemente salvata in `cachedSearchView` viene ripristinata immediatamente come contenuto della scena corrente, senza rieseguire alcuna query verso il database.

Considerazioni finali: l'architettura descritta, pur non essendo distribuita, adotta comunque una netta separazione delle responsabilità tra presentazione (client), logica di accesso ai dati (server) e modelli condivisi (DTO), rendendo l'applicazione facilmente estendibile: ad esempio, il livello DAO potrebbe in futuro essere esposto dietro un vero endpoint di rete (REST o RMI) senza dover riscrivere la logica dei Controller JavaFX, che già oggi dipendono esclusivamente dall'interfaccia offerta da `TheKnifeDAO` e non direttamente da JDBC o da SQL.



STRUTTURA DELLE CARTELLE DEL PROGETTO

Organizzazione fisica

Il progetto è organizzato come segue:

- **src/main/java**: codice Java di applicazione;
- **src/main/resources**: risorse FXML, CSS e scriptSQL;
- **pom.xml**: e wrapper Maven **mvnw** per la compilazione e gestione delle dipendenze;
- **target**: output di build generato da Maven.

Cartelle principali

```

1 TheKnife/
2   pom.xml
3   src/
4       main/
5           java/
6               module-info.java
7               db/
8                   DatabaseInitializer.java
9                   DatabaseManager.java
10                  TheKnifeDAO.java
11          project/demo/
12              CustomerProfileController.java
13              FavoritesController.java
14              HomeController.java
15              HomeLoggedController.java
16              LoginController.java
17              MyRestaurantController.java
18              Navigator.java
19              NewRestaurantController.java
20              OwnerDashboardController.java
21              RestaurantDetailsController.java
22              ReviewsController.java
23              SearchController.java
24              TheKnifeApp.java
25              Utente.java
26              Ristorante.java
27          resources/
28              theknife_create_db.sql
29              ristoranti_clean.csv
30          project/demo/
31              style.css
32              LoggedUser/
33              NotLoggedUser/
34              OwnerUser/

```



Relazione tra risorse e codice

Il codice Java fa riferimento a file FXML presenti in `resources/project/demo/NotLoggedInUser`, `LoggedInUser` e `OwnerUser`. Il `Navigator` risolve dinamicamente i percorsi di vista per supportare la navigazione modulo clientcontestuale.

BASE DI DATI

Il sistema utilizza PostgreSQL come Database Management System (DBMS) relazionale. La scelta di un database SQL garantisce l'integrità referenziale e la persistenza robusta.

Entità identificate

TheKnife è una piattaforma per la ricerca e la gestione di ristoranti, ispirata a TheFork. Il sistema prevede tre tipologie di attori: utenti non registrati (*guest*), clienti registrati e gestori di ristoranti registrati.

Entità	Descrizione	Attributi principali
Utente	Persona registrata. Può avere ruolo <i>cliente</i> o <i>gestore</i> (generalizzazione parziale ed esclusiva).	nome, cognome, email, password, data_nascita, domicilio
Cliente	Specializzazione di Utente. Può scrivere recensioni e salvare preferiti.	(eredita da Utente)
Gestore	Specializzazione di Utente. Inserisce e gestisce ristoranti, risponde alle recensioni.	(eredita da Utente)
Ristorante	Locale inserito da un gestore, con caratteristiche geografiche e di servizio.	nome, nazione, città, indirizzo, lat, lon, prezzo, delivery, prenotazione, cucina
Recensione	Valutazione (stelle 1–5 + testo) lasciata da un cliente per un ristorante.	stelle, testo, data
Risposta	Risposta del gestore a una recensione. Al massimo una per recensione. Entità debole dipendente da Recensione.	testo, data

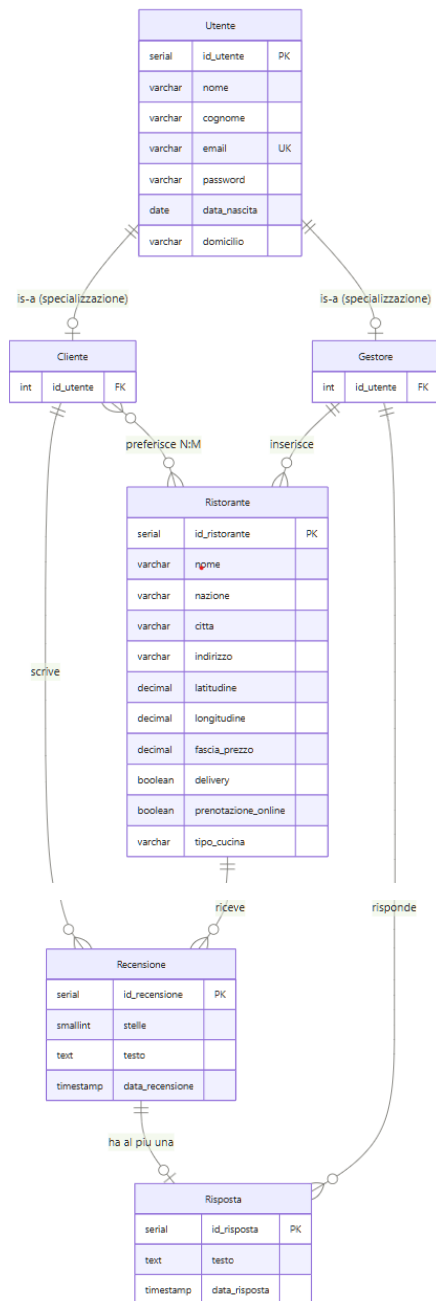
Relazioni e Cardinalità

Associazione	Entità coinvolte	Cardinalità	Note
Inserisce	Gestore → Ristorante	(1,N)	Un gestore inserisce uno o più ristoranti
Scrive	Cliente → Recensione	(1,N)	Un cliente può scrivere più recensioni
Riceve	Ristorante → Recensione	(1,N)	Un ristorante riceve più recensioni
Ha al più una risposta	Recensione → Risposta	(0,1)	Ogni recensione ha al massimo una risposta
Risponde	Gestore → Risposta	(1,N)	Solo il gestore del ristorante recensito risponde
Preferisce	Cliente ↔ Ristorante	(N:M)	Un cliente salva più ristoranti preferiti



Schema Concettuale ER (pre-ristrutturazione)

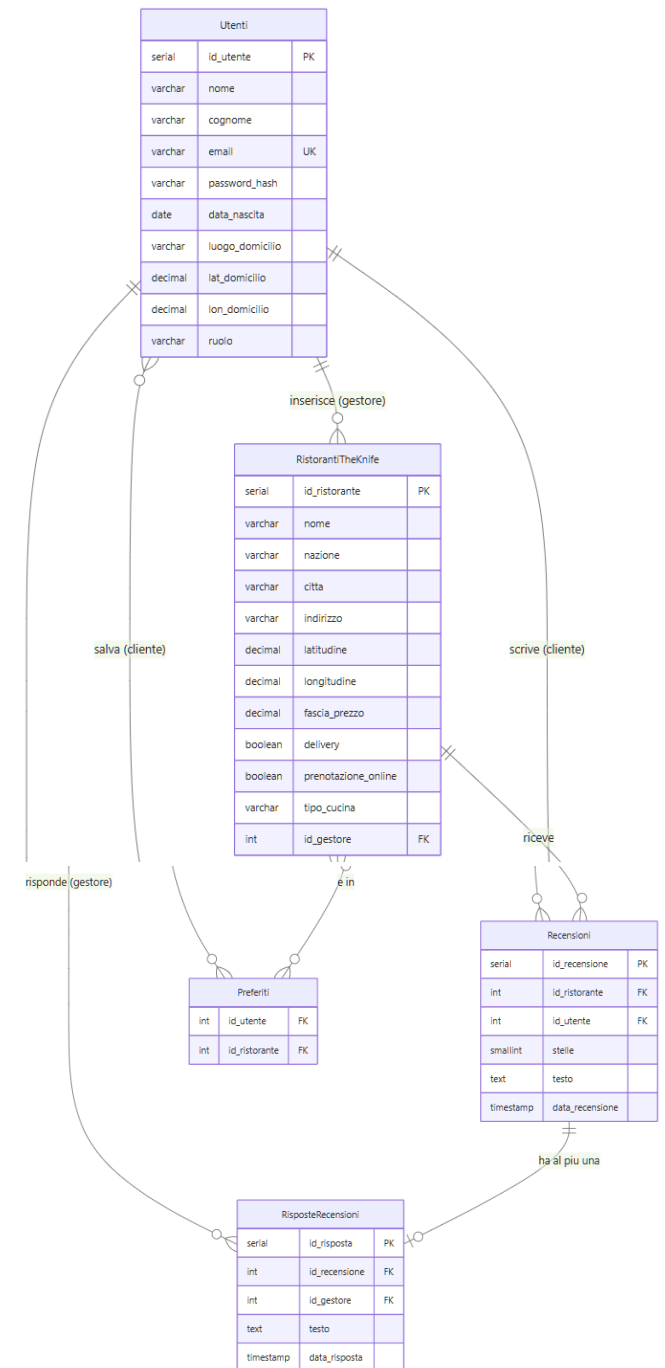
Il diagramma seguente rappresenta lo schema ER **prima della ristrutturazione**. La generalizzazione **Utente** → {**Cliente**, **Gestore**} è ancora esplicitata: le due specializzazioni sono entità distinte che ereditano gli attributi di Utente. La Risposta è modellata come entità debole identificata dalla Recensione cui appartiene.



□ Entità ||--|| Generalizzazione (is-a) ||--o{ Uno a molti ||--o{ Uno a zero/uno }o--o{ Molti a molti

Schema ER (post-ristrutturazione)

Il diagramma seguente rappresenta lo schema ER **dopo la ristrutturazione**. La generalizzazione è stata eliminata (Utenti unica tabella con discriminatore ruolo), RisposteRecensioni è diventata tabella autonoma, la relazione N:M è risolta nella tabella Preferiti.



□ Entità ||--o{ Uno a molti ||--o{ Uno a zero/uno }o--o{ Molti a molti (risolto)

Ristrutturazione dello Schema ER

La ristrutturazione trasforma lo schema ER eliminando generalizzazioni, entità deboli e risolvendo le relazioni N: M.

- **Eliminazione della generalizzazione Utente → {Cliente, Gestore}**

Prima

Superentità **Utente** con due sottotipi distinti: **Cliente** e **Gestore**. Generalizzazione parziale ed esclusiva.

I dati anagrafici di cliente e gestore sono identici. Non esistono attributi specifici per nessuno dei due sottotipi. Il collasso in un'unica tabella con discriminatore evita JOIN inutili durante il login e semplifica la registrazione, che è unica per entrambi i ruoli. La scelta "collasso verso il padre" è preferita rispetto a "collasso verso i figli" per evitare NULL multipli e tabelle ridondanti.

Dopo

Unica tabella **Utenti** con campo discriminatore ruolo ('cliente'/'gestore'). I vincoli sui ruoli sono garantiti dai trigger.

- **Eliminazione dell'entità debole Risposta**

Prima

Risposta era un'entità debole identificata dalla recensione cui appartiene. La cardinalità (0,1) garantiva al massimo una risposta per recensione.

La traduzione di un'entità debole richiede una chiave primaria composta difficile da gestire via JDBC. La tabella autonoma con UNIQUE su **id_recensione** garantisce lo stesso vincolo in modo più chiaro e con migliori prestazioni nelle query.

Dopo

RisposteRecensioni è una tabella autonoma con chiave primaria propria e UNIQUE (**id_recensione**) che garantisce al massimo una risposta.

- **Risoluzione della relazione N:M**

La relazione molti-a-molti tra Cliente e Ristorante è tradotta nella tabella associativa **Preferiti** con chiave primaria composta (**id_utente**, **id_ristorante**). Questa struttura impedisce duplicati e supporta direttamente le operazioni **aggiungiPreferito()**, **rimuoviPreferito()** e **visualizzaPreferiti()**.

Schema Relazionale

Traduzione dello schema ER ristrutturato in schema relazionale.

```
Utenti(id_utente PK, nome, cognome, email UK, password_hash, data_nascita, luogo_domicilio, lat_domicilio, lon_domicilio, ruolo)

RistorantiTheKnife(id_ristorante PK, nome, nazione, citta, indirizzo, latitudine, longitudine, fascia_prezzo, delivery, prenotazione_online, tipo_cucina, id_gestore FK-Utenti)

Recensioni(id_recensione PK, id_ristorante FK-RistorantiTheKnife, id_utente FK-Utenti, stelle, testo, data_recensione, UNIQUE(id_ristorante, id_utente))

RisposteRecensioni(id_risposta PK, id_recensione FK-Recensioni UK, id_gestore FK-Utenti, testo, data_risposta)

Preferiti(id_utente PK+FK-Utenti, id_ristorante PK+FK-RistorantiTheKnife)
```

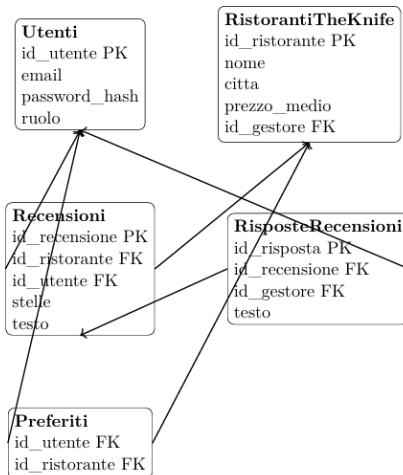


Tabelle dello schema relazionale

Utenti	RistorantiTheKnife	Recensioni	RisposteRecensioni
id_utente PK SERIAL	id_ristorante PK SERIAL	id_recensione PK SERIAL	id_risposta FK SERIAL
nome VARCHAR(100)	nome VARCHAR(255)	id_ristorante FK → RistorantiTheKnife	id_recensione FK UK → Recensioni
cognome VARCHAR(100)	nazione VARCHAR(100)	id_utente FK → Utenti	id_gestore FK → Utenti
email UK VARCHAR(255)	città VARCHAR(100)	stelle SMALLINT (1-5)	testo TEXT
password_hash VARCHAR(255)	indirizzo VARCHAR(255)	testo TEXT	data_risposta TIMESTAMP
data_nascita OPT DATE	latitudine DECIMAL(9,6)	data_recensione TIMESTAMP	UNIQUE (id_recensione) → max 1 risposta
luogo_domicilio VARCHAR(255)	longitudine DECIMAL(9,6)		
lat_domicilio OPT DECIMAL(9,6)	fascia_prezzo DECIMAL(8,2)		
lon_domicilio OPT DECIMAL(9,6)	delivery BOOLEAN		
ruolo VARCHAR(10)	prenotazione_online BOOLEAN		
	tipo_cucina VARCHAR(100)		
	id_gestore FK → Utenti		

Preferiti
id_utente FK FK → Utenti
id_ristorante FK FK → RistorantiTheKnife
PK composta (id_utente, id_ristorante)

Schema relazionale



Creazione dello schema

Lo script `theknife_create_db.sql` definisce le tabelle e i vincoli. Ecco un estratto significativo.

```

1 CREATE TABLE Utenti (
2   id_utente SERIAL PRIMARY KEY,
3   nome VARCHAR(100) NOT NULL,
4   cognome VARCHAR(100) NOT NULL,
5   email VARCHAR(255) NOT NULL UNIQUE,
6   password_hash VARCHAR(255) NOT NULL,
7   ruolo VARCHAR(10) NOT NULL,
8   data_nascita DATE,
9   luogo_domicilio VARCHAR(255) NOT NULL,
10  lat_domicilio DECIMAL(9, 6),
11  lon_domicilio DECIMAL(9, 6)
12 );
13
14 CREATE TABLE RistorantiTheKnife (
15   id_ristorante VARCHAR(50) PRIMARY KEY,
16   nome VARCHAR(255) NOT NULL,
17   nazione VARCHAR(100) NOT NULL,
18   città VARCHAR(100) NOT NULL,
19   prezzo_medio INT NOT NULL,
20   id_gestore INT NULL,
21
22   delivery BOOLEAN NOT NULL DEFAULT FALSE,
23   prenotazione_online BOOLEAN NOT NULL DEFAULT FALSE,
24   FOREIGN KEY (id_gestore) REFERENCES Utenti(id_utente)
25 );
  
```

Vincoli e integrità dei dati

Per garantire la robustezza del Data Tier, sono stati implementati i seguenti vincoli:

- Vincoli Foreign Key (FK)** sono 7 in totale:

Tabella	Campo FK	Riferimento	ON DELETE	ON UPDATE
RistorantiTheKnife	id_gestore	Utenti(id_utente)	RESTRICT	CASCADE
Recensioni	id_ristorante	RistorantiTheKnife(id_ristorante)	CASCADE	CASCADE
Recensioni	id_utente	Utenti(id_utente)	CASCADE	CASCADE
RisposteRecensioni	id_recensione	Recensioni(id_recensione)	CASCADE	CASCADE
RisposteRecensioni	id_gestore	Utenti(id_utente)	RESTRICT	CASCADE
Preferiti	id_utente	Utenti(id_utente)	CASCADE	CASCADE
Preferiti	id_ristorante	RistorantiTheKnife(id_ristorante)	CASCADE	CASCADE



RESTRICT su `id_gestore` in `RistorantiTheKnife` — se si tenta di eliminare un gestore che ha ristoranti attivi, il DB blocca l'operazione. È una scelta di sicurezza: un ristorante non può restare senza proprietario.

CASCADE su tutto il resto — se si elimina un ristorante, vengono eliminate automaticamente le sue recensioni e le relative risposte. Se si elimina un utente, spariscono le sue recensioni e i suoi preferiti. Mantiene la consistenza del DB senza dati orfani.

- **Unique Constraints** applicate a:
 - `Utenti.email`: Garantisce che ogni utente abbia un'e-mail univoca, che funge da username per il login.
 - `Recensioni(id_ristorante, id_utente)`: Garantisce che un cliente possa scrivere al massimo una recensione per ristorante.
 - `RisposteRecensioni.id_recensione`: Garantisce al massimo una risposta per ogni recensione.
 - `idx_utenti_email`: Indice unico sulla e-mail per rendere veloce il login. Tecnicamente ridondante ma lo abbiamo lasciato per chiarezza documentale.
- **Check Constraints:**
 - Usati in tabella `Utenti`, `RistorantiTheKnife`, `Recensioni`. In totale 7, uno per ogni vincolo di dominio esprimibile direttamente in PostgreSQL.
- **Vincoli procedurali (Trigger)**

Alcuni vincoli non sono esprimibili con CHECK standard in PostgreSQL poiché richiedono sub-query su altre tabelle. Vengono implementati tramite trigger BEFORE INSERT OR UPDATE.

Rif.	Trigger	Tabella	Descrizione
V1	tr_check_ruolo_gestore	RistorantiTheKnife	<code>id_gestore</code> deve riferirsi a un utente con ruolo = 'gestore'
V2	tr_check_ruolo_cliente	Recensioni	<code>id_utente</code> deve riferirsi a un utente con ruolo = 'cliente'
V5	tr_check_gestore_risposta	RisposteRecensioni	Il gestore che risponde deve essere il gestore del ristorante recensito

Utilizzo delle basi di dati nel sistema

L'interazione con PostgreSQL avviene esclusivamente nel Modulo Server attraverso uno strato di astrazione DAO (Data Access Object).

La classe richieste DB centralizza tutta la logica SQL. Utilizza il driver JDBC per stabilire connessioni sicure e gestire il ciclo di vita delle istruzioni:

- Connection Pooling: Gestisce l'apertura e la chiusura delle connessioni per ottimizzare le risorse.
- Prepared Statements: Tutte le query sono parametrizzate per prevenire attacchi di SQL Injection, garantendo che i dati inseriti dagli utenti non possano alterare la logica del database.

Per mantenere il codice pulito e modulare, le stringhe SQL sono organizzate in una classe dedicata (Queries). Questo permette di:

- Modificare la struttura delle query (es. ottimizzazione di una JOIN) senza dover intervenire nella logica di business del server.
- Mantenere una visione d'insieme di tutti gli endpoint del database, dai filtri di ricerca per autore alla generazione della media voti.



STRUTTURE DATI

Nella scelta delle strutture dati per il nostro **TheKnife** (è una piattaforma per la ricerca di ristoranti), abbiamo cercato il miglior equilibrio possibile tra velocità del software e semplicità di sviluppo.

Componenti Principali:

- **TheKnife** (Main/Controller): Gestisce l'interfaccia utente e il flusso dell'applicazione
- **GestoreRistoranti**: Gestisce la collezione di ristoranti e le operazioni di ricerca
- **GestoreUtenti**: Gestisce la collezione di utenti e l'autenticazione
- **UtenteCorrente**: Mantiene lo stato dell'utente loggato

Entità del Dominio:

- **Utente**: Contiene informazioni personali, credenziali, preferiti e ruolo (Cliente/Ristoratore)
- **Ristorante**: Contiene dati geografici, caratteristiche, recensioni e proprietario
- **Recensione**: Contiene valutazione, testo, data e risposta del ristorante
- **FiltroRicerca**: Oggetto per incapsulare i criteri di ricerca

Organizzazione dei Dati:

- **Liste**: Utilizzate per memorizzare collezioni di ristoranti, utenti e recensioni
- **Associazioni**: Ristorante-Proprietario, Utente-Preferiti, Ristorante-Recensioni

SCELTE ALGORITMICHE

1. Ricerca e Filtraggio

- **Algoritmo**: Ricerca lineare con filtri multipli
- **Complessità**: $O(n)$ dove n è il numero di ristoranti
- **Implementazione**: Scansione sequenziale applicando filtri per tipo cucina, prezzo, servizi

2. Ricerca Geografica

- **Algoritmo**: Confronto per nome città (ricerca per prossimità geografica)
- **Complessità**: $O(n)$ per la ricerca per località
- **Limitazione**: Non implementa calcolo di distanza reale basato su coordinate

3. Autenticazione

- **Algoritmo**: Ricerca lineare nella lista utenti
- **Complessità**: $O(n)$ per ogni tentativo di login
- **Sicurezza**: Hashing delle password tramite MessageDigest

4. Gestione Recensioni

- **Algoritmo**: Ricerca lineare per utente specifico
- **Complessità**: $O(r)$ dove r è il numero di recensioni per ristorante
- **Calcolo media**: Ricalcolo dinamico della media stelle

5. Persistenza Dati



- **Algoritmo:** Serializzazione/deserializzazione completa
- **Complessità:** $O(n)$ per caricamento e salvataggio
- **Approccio:** Caricamento all'avvio, salvataggio alla chiusura

Ottimizzazioni Possibili:

- **HashMap** per ricerche $O(1)$ su ID/username
- **Indici geografici** per ricerche spaziali efficienti
- **Caching** per risultati di ricerca frequenti
- **Lazy loading** per dati non immediatamente necessari

Permette una complessità temporale lineare in cambio di una implementazione diretta e mantenibile.

TEST ESEGUITI

- **Strategia:** nessun test unitario automatizzato; verifica tramite testing manuale del flusso applicativo.
- **Scenari testati:** Sono stati eseguiti i seguenti scenari:
 - registrazione e login di un cliente;
 - login di un gestore e creazione di un ristorante;
 - ricerca ristoranti con filtri combinati e ordini diversi;
 - inserimento, modifica ed eliminazione di recensioni;
 - aggiunta e rimozione di preferiti;
 - risposta alle recensioni da parte del gestore;
 - navigazione tra pagine e ritorno allo stato di ricerca salvato.
- **Test di persistenza:** Sono stati verificati:
 - prima inizializzazione automatica del database con `theknife_create_db.sql`;
 - importazione dati CSV con sanitizzazione e batch insert;
 - integrità referenziale con cancellazioni a cascata in **Recensioni** e **Preferiti**.
- **Osservazioni:** gestione corretta di email duplicata, assenza di login, recensioni duplicate; mancano test automatizzati JUnit.

POSSIBILI SVILUPPI FUTURI

- **Miglioramenti prioritari:**
 - implementazione dell'hashing password con BCrypt;
 - supporto di autenticazione sia sicura che estesa con sessioni crittografate;
 - aggiunta di test unitari e di integrazione;
 - estensione a un'architettura a servizi con API REST per la persistenza;
 - integrazione di mappe geografiche reali e geocoding per coordinate.
- **Estensioni funzionali:**
 - sistema di raccomandazione basato sui preferiti e sulle recensioni;
 - supporto multi-lingua e internazionale;
 - pagine FXML dedicate per la gestione avanzata del ristorante;



- miglioramenti UX con animazioni e feedback visivi;
- caching lato client delle query di ricerca.
- **Architettura futura:** separare la persistenza in repository/service layer dedicati; prevedere un API gateway per un'eventuale evoluzione cloud.

SITOGRAFIA/BIBLIOGRAFIA

Sitografia:

- Oracle Java Documentation: <https://docs.oracle.com/javase/8/docs/>
- JavaFX Documentation: <https://openjfx.io/documentation/>
- PostgreSQL Documentation: <https://www.postgresql.org/docs/>
- JDBC Documentation: <https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/>
- Maven Documentation: <https://maven.apache.org/docs/>

Bibliografia:

- Deitel, P. J., Deitel, H. M., Java: How to Program, Pearson.
- Schildt, H., Java: The Complete Reference, McGraw-Hill.
- Eckel, B., Thinking in Java, Prentice Hall.
- PostgreSQL Official Documentation, Manuale ufficiale del database PostgreSQL.
- JavaFX 17+ Documentation, guida ufficiale per lo sviluppo GUI.

